

Structural, Logical Sanity and Specification-Alignment Constraints

We define three sets of constraints to guide the validation of process models.

The first set, Structural Constraints (**STC**), checks whether a model follows the DSL grammar and its structural rules. Constraints **STC₁–STC₁₀** define the basic structure, including unique identifiers, node types, initial and end nodes, and the required numbers of incoming and outgoing transitions. Constraints **STC₁₁–STC₂₂** check variable domains, transitions, guards, labels, connectivity, and actor assignments. Together, they ensure that nodes are connected, guards and labels refer to existing transitions, variables are declared, and each node is assigned to exactly one actor.

The second set, Logical Sanity Constraints (**LSC**), checks the internal consistency of guard conditions. Constraints **LSC₁–LSC₃** require guards to be satisfiable within their declared variable domains, mutually exclusive for XOR branches, and collectively exhaustive for XOR and OR branches.

The third set, Specification-Alignment Alignment Constraints (**SPC**), checks whether the model accurately represents the source description. Constraints **SPC₁–SPC₇** address the completeness and accuracy of actions, execution order, control flow, variables, and actor assignments.

Below, we provide textual definitions of all constraints and logical formalizations of the structural and logical sanity constraints. These formal constraints apply to the full DSL representation.

Notation

Node and branching types. Let

$$\mathcal{K} = \{init, end, action, branch, converge\}$$

be the set of node types that accept a unique identifier, and let

$$T = \{AND, XOR, OR\}$$

be the set of branching types.

For convenience, $type(e, k)$ means that node e is declared with type $k \in \mathcal{K}$, regardless of any additional parameters in its declaration. For example,

$$type(e, action) \leftrightarrow \exists str \in L : action(e, str).$$

Guard expressions and evaluation. Let Φ be the set of syntactically valid Boolean expressions generated by the DSL grammar, and let $vars(\phi)$ denote the variables referenced by expression ϕ .

A state σ assigns each variable a value within its declared domain $\mathcal{R}_v$. We use $\mathcal{D}_\phi$ to denote all possible states of the variables referenced by guard ϕ , and $\mathcal{D}_e$ to denote all possible states of the variables referenced by the outgoing guards of branch e .

The function $eval(\phi, \sigma)$ returns *True* if ϕ holds under state σ , and *False* otherwise.

Paths and reachability. Let $k = |E|$. A path of length one is a direct transition:

$$path_1(x, y) \leftrightarrow precedence(x, y).$$

Paths of greater length are defined recursively:

$$path_n(x, y) \leftrightarrow \exists z \in E : path_{n-1}(x, z) \wedge precedence(z, y), \\ 2 \leq n < k.$$

For distinct nodes $x, y \in E$, reachability is defined as

$$path(x, y) \leftrightarrow \bigvee_{i=1}^{k-1} path_i(x, y).$$

Paths of length at most $k - 1$ are sufficient to determine whether one distinct node is reachable from another.

Structural Constraints

The Structural Constraints (**STC**) define the required structure and connectivity of the process model, together with the grammar specified in the **DSL-Grammar** file. They do not require interpretation of the source description.

- **STC₁**: Every atomic node, variable, and actor is represented by a strictly unique identifier. As E , V , and A are formally defined as mathematical sets, the uniqueness of identifiers within each specific set is inherently guaranteed. These three sets are also pairwise disjoint:

$$E \cap V = \emptyset \wedge E \cap A = \emptyset \wedge V \cap A = \emptyset$$

- **STC₂**: Every node $e \in E$ has exactly one assigned type.

$$\forall e \in E, \exists! K \in \mathcal{K} : K(e)$$

- **STC₃**: There is exactly one initial node per process model.

$$\exists!e \in E : \text{init}(e)$$

- **STC₄**: The initial node cannot have incoming transitions.

$$\forall e \in E \left(\text{init}(e) \rightarrow \neg \exists x \in E : \text{precedence}(x, e) \right).$$

- **STC₅**: There is at least one end node per process model.

$$\exists e \in E : \text{end}(e)$$

- **STC₆**: End nodes cannot have outgoing transitions.

$$\forall e \in E \left(\text{end}(e) \rightarrow \neg \exists x \in E : \text{precedence}(e, x) \right).$$

- **STC₇**: The descriptions of actions, actors, and variables must not be empty. Let ϵ denote the empty string. Let E_a denote to the set of nodes listed in the actor declaration:

$$\forall e \in E, \forall str \in L \left(\text{action}(e, str) \rightarrow str \neq \epsilon \right).$$

$$\forall a \in A, \forall str \in L, \forall E_a \subseteq E \left(\text{actor}(a, str, E_a) \rightarrow str \neq \epsilon \right).$$

$$\forall v \in V, \forall str \in L, \forall \mathcal{R}_v \left(\text{defines}(v, str, \mathcal{R}_v) \rightarrow str \neq \epsilon \right).$$

- **STC₈**: Each action node has exactly one incoming and one outgoing transition.

$$\forall e \in E \left(\text{type}(e, \text{action}) \rightarrow [(\exists!p \in E : \text{precedence}(p, e)) \wedge (\exists!s \in E : \text{precedence}(e, s))] \right).$$

- **STC₉**: Each branch node have exactly one incoming and at least two outgoing transitions.

$$\forall e \in E \left(\text{type}(e, \text{branch}) \rightarrow [(\exists!p \in E : \text{precedence}(p, e)) \wedge (\exists s_1, s_2 \in E : s_1 \neq s_2 \wedge \text{precedence}(e, s_1) \wedge \text{precedence}(e, s_2))] \right).$$

- **STC₁₀**: Each converge node must have at least two incoming transitions and exactly one outgoing transition.

$$\forall e \in E \left(\text{converge}(e) \rightarrow [(\exists p_1, p_2 \in E : p_1 \neq p_2 \wedge \text{precedence}(p_1, e) \wedge \text{precedence}(p_2, e)) \wedge (\exists!s \in E : \text{precedence}(e, s))] \right).$$

- **STC₁₁**: A defined variable's range must contain at least two distinct values.

$$\forall v \in V, \forall str \in L, \forall \mathcal{R}_v \left(\text{defines}(v, str, \mathcal{R}_v) \rightarrow |\mathcal{R}_v| \geq 2 \right).$$

- **STC₁₂**: A precedence relationship requires both source and target nodes to be elements of E .

$$\forall x, y \left(\text{precedence}(x, y) \rightarrow x \in E \wedge y \in E \right).$$

- **STC₁₃**: A guard with condition ϕ can only be placed on an existing transition.

$$\forall e, s \in E, \forall \phi \in \Phi \left(\text{guard}(e, s, \phi) \rightarrow \text{precedence}(e, s) \right)$$

- **STC₁₄**: A guard condition must be a syntactically valid boolean expression.

$$\forall e, s \in E, \forall \phi \left(\text{guard}(e, s, \phi) \rightarrow \phi \in \Phi \right)$$

- **STC₁₅**: Let $vars(\phi)$ denote the set of variables present in expression ϕ . Every variable in ϕ must be formally defined.

$$\forall e, s \in E, \forall \phi \in \Phi \left(guard(e, s, \phi) \rightarrow \forall v \in vars(\phi), \right. \\ \left. \exists str \in L, \exists \mathcal{R}_v : defines(v, str, \mathcal{R}_v) \right).$$

- **STC₁₆**: Each outgoing transition from an XOR or OR branch must have exactly one guard:

$$\forall e, s \in E \left((\exists t \in \{XOR, OR\} : branch(e, t) \wedge precedence(e, s)) \rightarrow \exists! \phi \in \Phi : guard(e, s, \phi) \right)$$

- **STC₁₇**: Guard conditions can only originate from XOR or OR branches.

$$\forall e, s \in E, \forall \phi \in \Phi \left(guard(e, s, \phi) \rightarrow \exists t \in \{XOR, OR\} : branch(e, t) \right)$$

- **STC₁₈**: A textual label can only be attached to an existing transition.

$$\forall e, s \in E, \forall str \in L \left(label(e, s, str) \rightarrow precedence(e, s) \right)$$

- **STC₁₉**: Every node must be reachable from the initial node.

$$\forall e \in E \left(\neg init(e) \rightarrow \exists p \in E : init(p) \wedge path(p, e) \right).$$

- **STC₂₀**: Every node must have a path to an end node.

$$\forall e \in E \left(\neg end(e) \rightarrow \exists s \in E : end(s) \wedge path(e, s) \right).$$

- **STC₂₁**: Each declared actor must be assigned at least one node. Here, E_a denotes the set of nodes listed in the actor declaration:

$$\forall a \in A, \forall str \in L, \forall E_a \subseteq E \left(actor(a, str, E_a) \rightarrow E_a \neq \emptyset \right).$$

- **STC₂₂**: Each node must be assigned to exactly one declared actor through the actor-assignment relation $S \subseteq A \times E$:

$$\forall e \in E, \exists! a \in A : (a, e) \in S.$$

Logical Sanity Constraints

The Logical Sanity Constraints (**LSC**) check the consistency of guard conditions using the expressions and variable domains defined in the DSL. They do not require interpretation of the source description.

- **LSC₁**: Each guard must evaluate to *True* for at least one state within the declared domains of its variables. An unsatisfiable guard defines a transition that can never be taken:

$$\forall e, s \in E, \forall \phi \in \Phi \left(guard(e, s, \phi) \rightarrow \exists \sigma \in \mathcal{D}_\phi : eval(\phi, \sigma) \right).$$

- **LSC₂**: The guards of distinct outgoing transitions from an XOR branch must not evaluate to *True* in the same state. This ensures that at most one outgoing transition is enabled:

$$\forall e \in E \left(branch(e, XOR) \rightarrow \forall \sigma \in \mathcal{D}_e, \forall s_1, s_2 \in E, \forall \phi_1, \phi_2 \in \Phi \right. \\ \left. \left[(s_1 \neq s_2 \wedge guard(e, s_1, \phi_1) \wedge guard(e, s_2, \phi_2)) \right. \right. \\ \left. \left. \rightarrow \neg (eval(\phi_1, \sigma) \wedge eval(\phi_2, \sigma)) \right] \right).$$

- **LSC₃**: The outgoing guards of each XOR or OR branch must cover the full joint domain of their variables. For every state, at least one guard must evaluate to *True*. This prevents the branch from being blocked because no condition is satisfied:

$$\forall e \in E \left((branch(e, XOR) \vee branch(e, OR)) \rightarrow \forall \sigma \in \mathcal{D}_e, \right. \\ \left. \exists s \in E, \exists \phi \in \Phi : guard(e, s, \phi) \wedge eval(\phi, \sigma) \right).$$

Specification-Alignment Constraints

The Specification-Alignment Constraints (**SPC**) check whether the process model preserves the meaning and behavior of the natural-language description. Unlike the structural and logical sanity constraints, these checks require contextual interpretation of the source text. We therefore express them informally.

- **SPC₁**: The process model must capture all documented behaviors from the source text. There must be no omissions of described actions, nor any hallucinated additions.
- **SPC₂**: Each node must strictly preserve the original meaning of its corresponding text. Internal payloads (action strings, logic variables, values, and units) must remain factually accurate to the source without any modification.
- **SPC₃**: The precedence relationships must preserve the chronological sequence of actions exactly as dictated by the source narrative, without mutating, addition, omission or reordering the steps.
- **SPC₄**: The process model must correctly resolve and represent all control flows and branching logic implied by the text, including implicit cycles, simultaneous tasks, and exclusive or inclusive conditional decision boundaries.
- **SPC₅**: Every sequence of connected relationships from the initial node to an end node must exactly mirror an intended progression of events described in the input process. There must be no omissions of described flows, nor any hallucinated additions.
- **SPC₆**: The definition of variables, and their acceptable range must faithfully reflect the process description and range described in the source text. Variables must not be hallucinated, described conditions must not be omitted, and their range not be mutated from the process description.
- **SPC₇**: The assignment of process nodes to actors must correctly reflect the allocation of responsibilities, and participants as described in the source process description. There must be no tasks assigned to the incorrect participant, nor any hallucination of omitted actors.